



카테고리 없음

# [논문 리뷰] Accurate predictions on small data with a tabular...

yuha933 2026. 4. 6. 18:30

## Introduction

### 연구 배경

- 수작업으로 설계된 알고리즘 구성 요소들은 더 높은 성능을 보이는 end-to-end 학습 방식으로 대체되어 왔다. 컴퓨터 비전에서는 SIFT와 HOG와 같은 수작업 특징들이 학습된 convolution으로 대체되었고, 자연어 처리에서의 문법 기반 접근 방식은 학습된 transformer로 대체되었다.
- 표형 데이터셋은 텍스트나 이미지와 같은 비가공 데이터 형태와 구별되는 다양한 특성을 가진다.
- 딥러닝 방법들은 전통적으로 표형 데이터에서 어려움을 겪어왔으며, 이는 데이터셋 간의 이질성과 원시 데이터 자체의 이질성 때문이다. 이러한 이유로 트리 기반 모델과 같은 비딥러닝 방법들이 지금까지 가장 강력한 경쟁자로 자리잡아 왔다.

### 기존 연구의 한계

- 전통적인 머신러닝 모델들은 아래와 같은 한계들을 지닌다.

- 충분한 전처리 없이 사용될 경우, out-of-distribution 데이터에 대한 예측 성능이 낮다.
- 한 데이터셋에서 다른 데이터셋으로 지식을 전이하는 능력이 부족하다.
- gradient를 전파하지 않기 때문에 신경망과 결합하기 어렵다.

## 본 연구의 제안

- 소규모에서 중간 규모의 표형 데이터를 위한 foundation model인 TabPFN을 제안한다.
  - 새로운 지도학습 기반 표형 학습 방법은 소규모에서 중간 규모의 어떤 데이터셋에서도 적용 가능하며, 최대 10,000개의 샘플과 500개의 feature를 가진 데이터셋에서 뛰어난 성능을 보인다.
  - 단 한 번의 forward pass만으로, TabPFN은 벤치마크에서 state-of-the-art 방법들, 특히 gradient-boosted DT보다도 훨씬 뛰어난 성능을 보인다.
  - fine-tuning, 생성 능력, 밀도 추정 등 다양한 foundation model의 특성을 가진다.

## Principled in-context learning

### ICL 도입 배경

- LLM이 잘 되는 이유는 ICL(in-context learning) 덕분이다. transformer의 경우에는 ICL로 로지스틱 회귀와 베이지안 모델까지 학습이 가능하다.
- 기존 TabPFN에서도 ICL을 도입했지만, 개념적으로만 가능하고 실적 적용이 어렵다는 문제가 있었다. 이에 개선된 TabPFN, 즉 본 연구에서는 더 큰 데이터셋 처리가 가능하도록, regression, categorical, missing value 지원이 가능하도록, 그리고 outlier나 irrelevant feature에 강건하도록 개선하였다.

### 본 연구의 아이디어

- 기존 연구들은 사람이 알고리즘을 설계하는 방식이었지만, TabPFN은 모델이 데이터 예시를 통해 알고리즘을 학습하는 exemplar-based declarative programming 방식을 사용한다. 이러한 방식은 forward pass 한 번으로 gradient와 retraining 없이 학습과 예측을 동시에 가능하도록 한다.
- 이때, 데이터셋은 다양한 tabular 데이터셋을 인위적으로 대량 생성해서 사용한다. 이러한 데이터셋은 feature-target 관계가 다양하게, 노이즈나 결측값, 이상치는 포함하도록 구성된다.

### 전체 파이프라인

1. Data Generation : 다양한 synthetic dataset을 생성하고, 일부 label을 masking한다.
2. Pre-training : transformer가 missing label을 예측하도록 학습한다.
3. Real-world Prediction : 새로운 dataset을 입력하고, 즉시 예측하는 ICL 방식을 가능하게 한다.

## 이론적 해석

$$p(y_{\text{test}}|X_{\text{test}}, X_{\text{train}}, y_{\text{train}})$$

- train 데이터를 기반으로 test를 예측하도록 학습하며, TabPFN은 Bayesian inference 근사를 하는 것으로 해석할 수 있다.

## An architecture designed for tables

### 문제 1 : transformer와 tabular 데이터 구조가 안 맞는다.

- 문제 : transformer는 sequence용 구조이기 때문에 tabular 데이터에 사용하면 데이터를 표 구조(행/열)로 보는 것이 아니라 일렬(문장처럼)로 본다.
- 본 연구의 아이디어 : 표를 sequence로 보지 말고, 각 cell에 개별 representation을 부여해서 cell들의 집합으로 본다.
- Two-way attention 구조 : 각 cell이 보는 방향이 2개라서, 결과적으로 feature 관계와 데이터 분포 모두 학습할 수 있다.
  - Row 방향 : 같은 데이터 안에서 feature 간 관계를 학습한다.
  - Column 방향 : 다른 샘플 간 비교를 학습한다.

### 문제 2 : 계산 낭비 문제가 발생한다.

- 문제 : 기존 방식은 test마다 train을 다시 계산하는 구조다보니 계산 낭비가 생긴다.
- 본 연구의 아이디어 : train으로 ICL을 한 번 수행한 후, 그 결과를 저장해둔 다음 여러 test에 재사용한다.
- 결과 : CPU 환경에서는 최대 800배 속도 향상을, GPU에서는 최대 30배 속도 향상을 보였다.

## 추가 최적화

- flash attention, half precision, activation checkpointing을 효율성을 위해 추가적으로 적용하였다.
- 결과 : 메모리가 1/4로 감소하였으며, 큰 데이터도 처리 가능함을 확인하였다.

## Regression 문제 처리 방식 변경

- 기존 연구에서는 하나의 값을 예측했다면, TabPFN에서는 확률 분포를 예측하는 방식으로 변경하였다.

## Synthetic data based on causal models

### synthetic data의 도입

- 문제 : 현실 데이터는 부족하고, 편향이 존재하며, 개인정보나 저작권 문제가 있는 경우가 많다. 그러나 TabPFN 성능의 핵심은 좋은 학습 데이터이다.
- 본 연구의 아이디어 : 데이터 자체를 만들자 !!
- 단순 랜덤 데이터가 아니라 SCM(Structural Casual Model)을 사용하여 feature들이 어떻게 서로 영향을 주는지 원인-결과 관계를 가진 데이터를 생성한다.

### 전체 생성 파이프라인

1. Hyperparameter sampling : 데이터 크기, feature 개수와 난이도 등 어떤 문제를 만들지를 먼저 결정한다.
2. Causal graph construction : DAG(Directed Acyclic Graph)를 생성하여 feature들 사이 관계 구조를 정의한다.
3. Data propagation : 초기값(noise)을 생성한 후, graph를 따라 값을 전달한다. 이때 각 edge마다 neural network, activation, DT 구조, categorical 변환, Gaussian noise 추가를 하며, 현실처럼 복잡한 데이터를 생성한다.

→ 결과적으로, graph 끝까지 지나면 feature와 target이 생성된다.

### Post-processing

- Kumaraswamy distribution
- warping
- discretization

### 최종 결과

- 약 1억개의 synthetic dataset을 생성하였으며, 각 dataset은 다른 구조, 다른 feature, 다른 관계를 가지는 dataset이다.

## Qualitative analysis

## 분석 목적

1. TabPFN이 어떤 상황에서 어떻게 동작하는지 직관적으로 이해하기 위해서
2. 다양한 데이터 특성이 모델에 미치는 영향을 분리해서 보기 위해

## 기존 모델 vs. TabPFN

| Model                       | 특징                                                                           | 단점                              |
|-----------------------------|------------------------------------------------------------------------------|---------------------------------|
| Linear Regression           | 선형 관계만 학습 가능하며, 단순하고 해석 가능하다.                                                | 비선형 데이터에서는 성능이 급격히 하락한다.        |
| MLP                         | 비선형 학습이 가능하다.                                                                | 불연속적이거나 급격한 변화에서는 성능이 떨어진다.     |
| CatBoost (Tree-based Model) | 구간별 함수로 모델링하며, 안정적이다. (catastrophic failure이 없다.)                            | 근사 오차가 존재하고, 예측이 직관적이지 않을 수 있다. |
| TabPFN                      | 부드러운 함수와 불연속 함수 모두 잘 처리한다. 또한, step function도 잘 근사하고 신경망인데도 불연속 패턴 대응이 가능하다. | -                               |

## TabPFN의 장점 : 불확실성 모델링이 가능하다.

- 기존 모델은 하나의 값만 출력했다면, TabPFN은 확률 분포를 출력해 이 값일 가능성이 어느 정도인지까지 같이 예측해준다.
- ex. double slit experiment (복잡한 multi-modal 분포 생성하는 실험)
  - TabPFN : 한 번의 연산으로 복잡한 분포 그대로 예측한다.
  - 기존 모델 (CatBoost) : 여러 모델을 따로 학습해야 하고, 분포를 나중에 재구성해야 한다. 이로 인해 시간이 오래 걸리고 성능 또한 떨어진다.

## Quantitative analysis

### 실험 배경

- 사용 데이터셋
  - AutoML Benchmark와 OpenML-CTR23을 사용하였으며, 추가로 Kaggle 대회 데이터와 Tabular Playground Series도 사용하였다.
  - 분류 29개, 회귀 28개로 구성되었으며 최대 10,000개의 샘플, 최대 500개의 feature로 구성된 데이터셋을 사용하였다.
- baseline

- 트리 기반으로는 Random Forest, XGBoost, CatBoost, LightGBM을 사용하였다.
- 그 외로 선형 모델과 SVM, MLP도 사용하였다.
- 평가 지표
  - 분류 지표로는 ROC-AUC와 Accuracy를, 회귀 지표로는  $R^2$ 과 RMSE를 사용하였다. 모든 결과는 정규화하여 최고 성능이면 1, 최악의 성능일 수록 0에 가깝게 보았다.
- 실험 설정
  - 각 실험은 10번 반복하였으며, random seed는 변경하였다.
  - 데이터 분할은 train : test = 9:1 로 하였다.
  - 하이퍼파라미터 튜닝으로는 random search, 5-fold cross-validation을 사용하였다.
- TabPFN은 사전학습은 GPU 8개로 2주 동안 한 번만 수행하였으며, 이후 새로운 데이터셋마다 추가 학습 없이 forward pass 1번으로 예측하였다.

## Comparison with state-of-the-art baselines

### 실험 목적

- 튜닝이 없어도 좋은가?

### default setting 결과

- 분류 : CatBoost보다 +0.187
- 회귀 : CatBoost보다 +0.051
- 결과적으로, 튜닝 없이도 이미 최고 수준이다.

### tuning 결과 비교

- 다른 모델들은 최대 4시간 tuning을 진행하였다.
- 분류 : +0.13
- 회귀 : +0.093
- 다른 모델들을 tuning하였음에도 TabPFN이 여전히 다른 모델보다 높은 성능을 보인다.

## 속도 비교

- TabPFN : 2.8초 / 4.8초
- 기존 모델 : 최대 4시간
- 분류에서는 5,140배, 회귀에서는 3,000배 가까이 빠른 것을 볼 수 있다.

## Evaluating diverse data attributes

### 실험 목적

- 데이터 특성이 바뀌면 성능이 어떻게 변하는가?

### 독립 변인

- useless feature 추가
- outliers 추가
- sample / feature 수 감소
- categorical / missing value 포함 여부

### 실험 결과

- noise(outliers) / useless feature → TabPFN은 매우 robust하지만, MLP는 성능이 크게 하락함을 보였다.
- sample / feature 수 감소 → 모든 모델 성능이 감소했지만, TabPFN은 절반 데이터에서도 여전히 상위 성능을 유지하였다.
- categorical / missing value 포함 여부 → TabPFN 성능에 거의 영향이 없다.

## Comparison with tuned ensemble methods

### 실험 대상

- 비교 대상 : AutoGluon (stacked ensemble + tuning)
- TabPFN 확장 : TabPFN끼리 ensemble + tuning을 했다.

### 실험 결과

- 분류 : AutoGluon보다 5,140배 더 빠르며 더 좋은 성능을 보였다.
- 회귀 : AutoGluon보다 48배 빠르면서, 성능도 더 높음을 보였다.

## Foundation model with interpretability

## Foundation model로써의 TabPFN 분석

- Foundation model은 여러 task에 활용 가능한 범용 모델을 말한다.
- Density estimation : 수치형 데이터의 경우에는 확률 밀도 함수를 추정하고, 범주형 데이터의 경우에는 확률 질량 함수를 추정한다. 이로 인해 TabPFN은 단순 예측이 아니라, 데이터가 얼마나 정상인지 판단 가능하다.
- Data generation : 실제 데이터처럼 생긴 synthetic tabular data를 생성하여, 데이터 부족을 해결, privacy 보호, 데이터 증강을 가능하게 한다.
- Representation learning : feature representation을 학습하여 결측값 보정과 클러스터링을 가능하게 한다. 이를 통해 raw 데이터보다 클래스 분리가 더 잘 되게 된다.
- Fine-tuning : 트리 모델과 다르게 fine-tuning이 가능해서 새로운 데이터에도 적응 가능하게끔 한다.
- Interpretability : SHAP을 사용해 각 feature가 예측에 얼마나 기여했는지를 계산한다.

## 기존 모델 vs. TabPFN

| Model               | 해석 난이도 | 성능 |
|---------------------|--------|----|
| Logistic regression | 낮음     | 낮음 |
| CatBoost            | 어려움    | 높음 |
| TabPFN              | 낮음     | 높음 |

## Conclusion

### Future work

- Scaling to larger datasets : 더 큰 규모의 데이터셋으로 확장하는 연구
- Handling data drift : 시간에 따라 데이터 분포가 변하는 문제(data drift)를 해결하는 방법 연구
- Fine-tuning across related tabular tasks : 서로 관련된 tabular 데이터셋 간에서 모델을 효과적으로 fine-tuning하는 방법 연구
- Understanding theoretical foundations : TabPFN이 왜 잘 작동하는지에 대한 이론적 기반을 더 깊이 이해하는 연구
- Extending to new data modalities : 다양한 데이터 유형으로 확장한 연구들 (시계열 데이터, 멀티모달 데이터, ECG, 신경영상 데이터, 유전체 데이터 등)
- Designing specialized priors : 데이터 유형별로 더 적합한 맞춤형 prior를 설계하는 연구